

SECURE DOCUMENT VAULT

Data Integrity and Authentication System

Final Project Report

May 20, 2026

Prepared by:

- **Ahmed Yasser** (ID: 2305127)
 - **Abdul Rahman Rabie** (ID: 2305098)
 - **Ahmed Alsayed** (ID: 2305121)
 - **Mostafa Hamdy** (ID: 2305021)
-

1. Executive Summary

This project implements a comprehensive Secure Document Vault System - a web-based platform that demonstrates industry-standard security practices for document management. The system integrates multiple modern cryptographic techniques and authentication mechanisms to ensure data confidentiality, integrity, and authenticity.

The application successfully implements:

- Multi-factor authentication with Two-Factor Authentication (2FA)
- Role-Based Access Control (RBAC) with three user roles (Admin, Manager, User)
- Document encryption using AES-256 algorithm
- Digital signatures and integrity verification using RSA and SHA-256
- OAuth integration with GitHub and Google
- HTTPS secure communication

- Comprehensive activity logging and audit trails

The project demonstrates that when properly implemented, modern security measures protect sensitive data against unauthorized access, tampering, and interception.

2. Project Overview

2.1 Objectives

- Understand and implement modern authentication mechanisms
- Apply secure password storage and password policy validation
- Implement Role-Based Access Control (RBAC)
- Secure APIs and protected resources using JWT authentication
- Integrate OAuth login using GitHub and Google
- Implement Two-Factor Authentication (2FA)
- Apply encryption techniques to protect uploaded documents
- Implement digital signatures and integrity verification
- Secure communication using HTTPS
- Demonstrate HTTPS protection through Wireshark analysis

2.2 Technology Stack

Component	Technology
Backend	Node.js + Express.js
Frontend	React + Tailwind CSS
Database	MySQL (XAMPP)
Authentication	JWT, bcryptjs, Passport.js
Encryption	Node.js crypto (AES-256, RSA)
2FA	TOTP (speakeasy)

3. System Architecture

The Secure Document Vault follows a three-tier architecture:

3.1 Frontend Layer (React)

- Login/Register pages with OAuth integration
- Dashboard for document management
- Document verification interface
- Settings page for 2FA management
- Admin panel for user management
- Real-time password policy validation

3.2 Backend Layer (Express.js)

- RESTful API endpoints with authentication middleware
- JWT token validation for protected resources
- Role-based authorization checks
- Document encryption/decryption operations
- Digital signature generation and verification
- Activity logging and audit trails

3.3 Data Layer (MySQL)

- Users and roles management
- Documents storage with metadata
- Encryption keys management
- Audit logs and activity history
- OAuth session storage

4. Security Features Implemented

4.1 Authentication Mechanisms

- **User Registration & Login:** Traditional email/password authentication with bcrypt hashing.
- **JWT Tokens:** Stateless authentication using JSON Web Tokens.
- **OAuth Integration:** GitHub and Google OAuth for social login.
- **Two-Factor Authentication:** Time-based One-Time Password (TOTP) using Google Authenticator.
- **Account Lockout:** Automatic lockout after 5 failed login attempts for 30 minutes.

4.2 Password Security

The system enforces strict password requirements:

- Minimum 12 characters length
- At least one uppercase letter (A-Z)
- At least one lowercase letter (a-z)
- At least one number (0-9)
- At least one special character (!@#\$%^&*)
- Hashed using bcrypt with salt rounds = 12
- Never stored in plain text in database

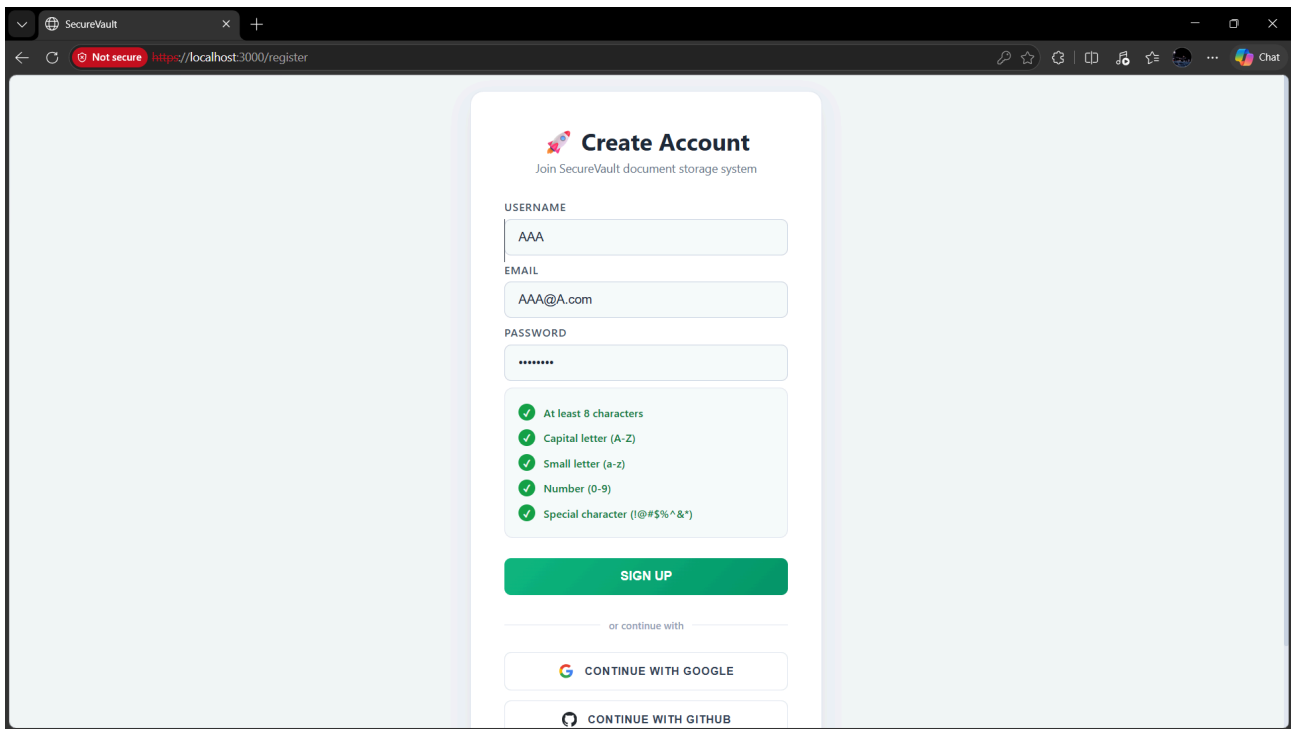


Figure 1: Password policy validation during registration.

4.3 Role-Based Access Control (RBAC)

Role	Permissions
Admin	Manage users, manage roles, view all documents, delete any document, verify documents
Manager	Review documents, verify documents, view assigned documents
User	Upload documents, download own documents, delete own documents, view own documents

5. Encryption & Data Protection

5.1 Document Encryption (AES-256)

- **Algorithm:** AES-256-CBC (Cipher Block Chaining)
- **Key Size:** 256 bits (32 bytes)
- **Initialization Vector:** Randomly generated 16 bytes

- **Operation:** Each document is encrypted with a unique key and random IV, both stored securely.

5.2 Digital Signatures (RSA-2048)

- **Algorithm:** RSA-2048 with SHA-256
- **Process:** A 2048-bit RSA key pair is used to sign the document hash, ensuring authenticity and non-repudiation.

5.3 Hashing (SHA-256)

- **Purpose:** Data integrity verification.
- **Algorithm:** SHA-256 (Secure Hash Algorithm 256-bit).
- **Usage:** Detects any tampering by comparing the current file hash with the original stored hash.

6. System Features & Functionality

6.1 Dashboard

The dashboard provides a centralized view of system statistics and active security features.

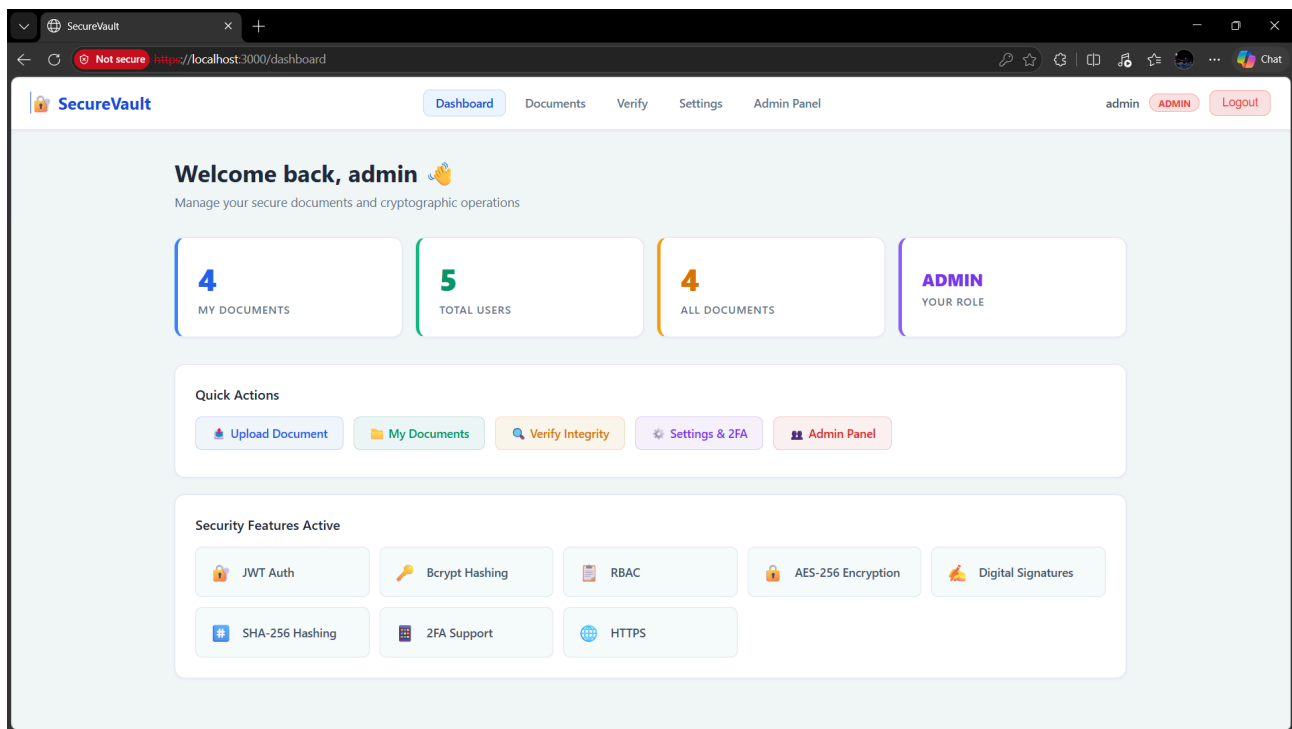


Figure 2: System Dashboard showing active security modules.

6.2 Document Management

Features include drag-and-drop upload, file type/size validation, and automatic encryption/decryption.

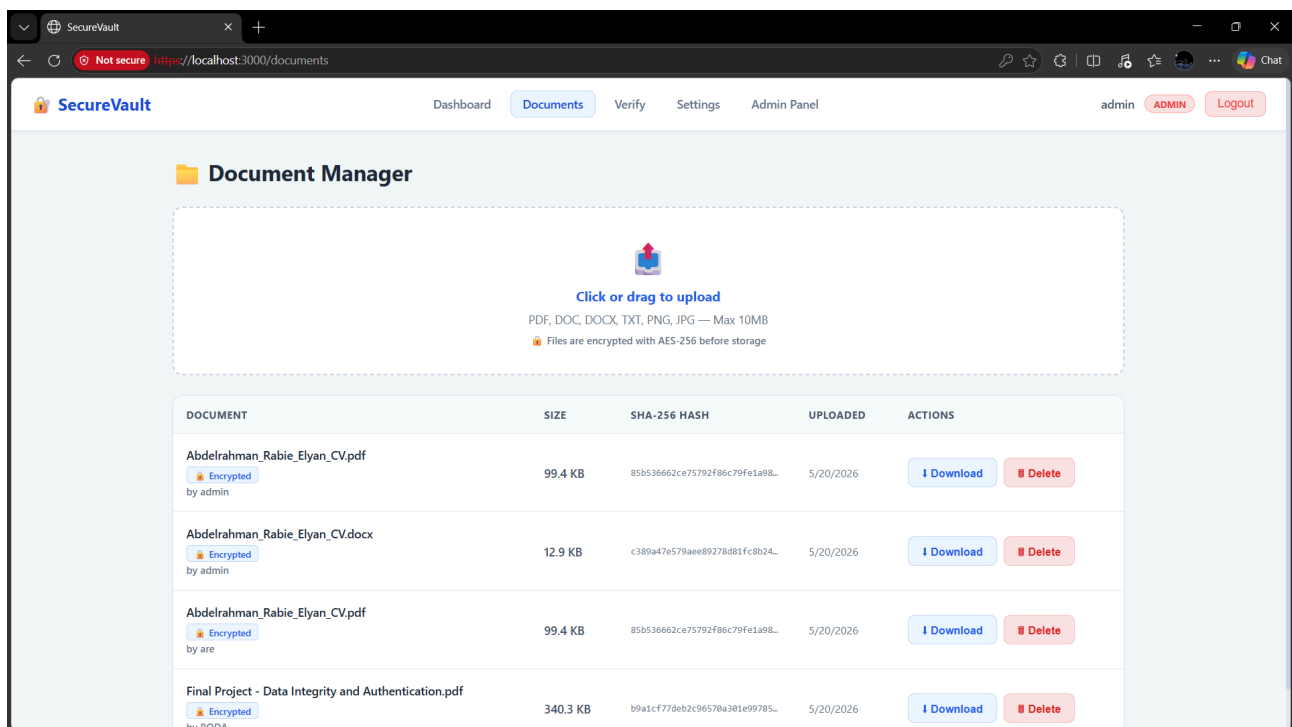


Figure 3: Document management interface with encryption status.

6.3 Document Integrity Verification

Users can verify that documents haven't been tampered with by checking hashes and digital signatures.

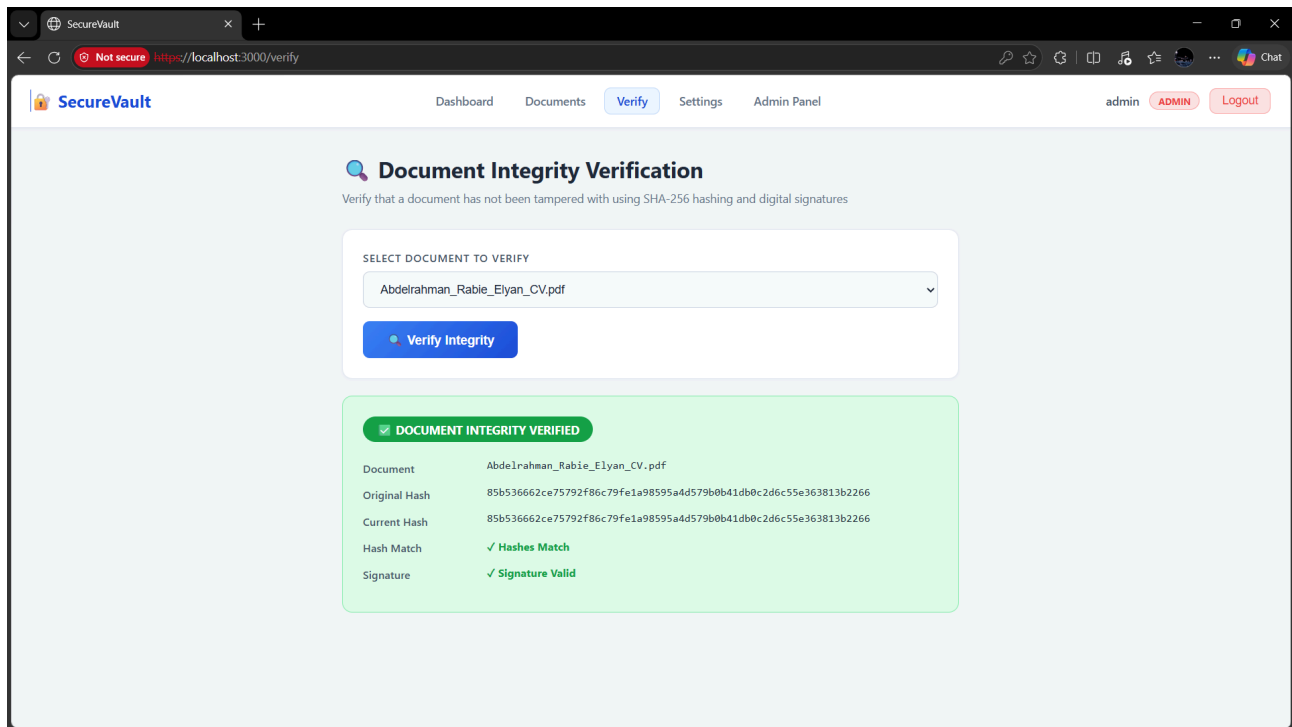


Figure 4: Successful document integrity and signature verification.

6.4 Two-Factor Authentication (2FA)

The system supports TOTP via apps like Google Authenticator.

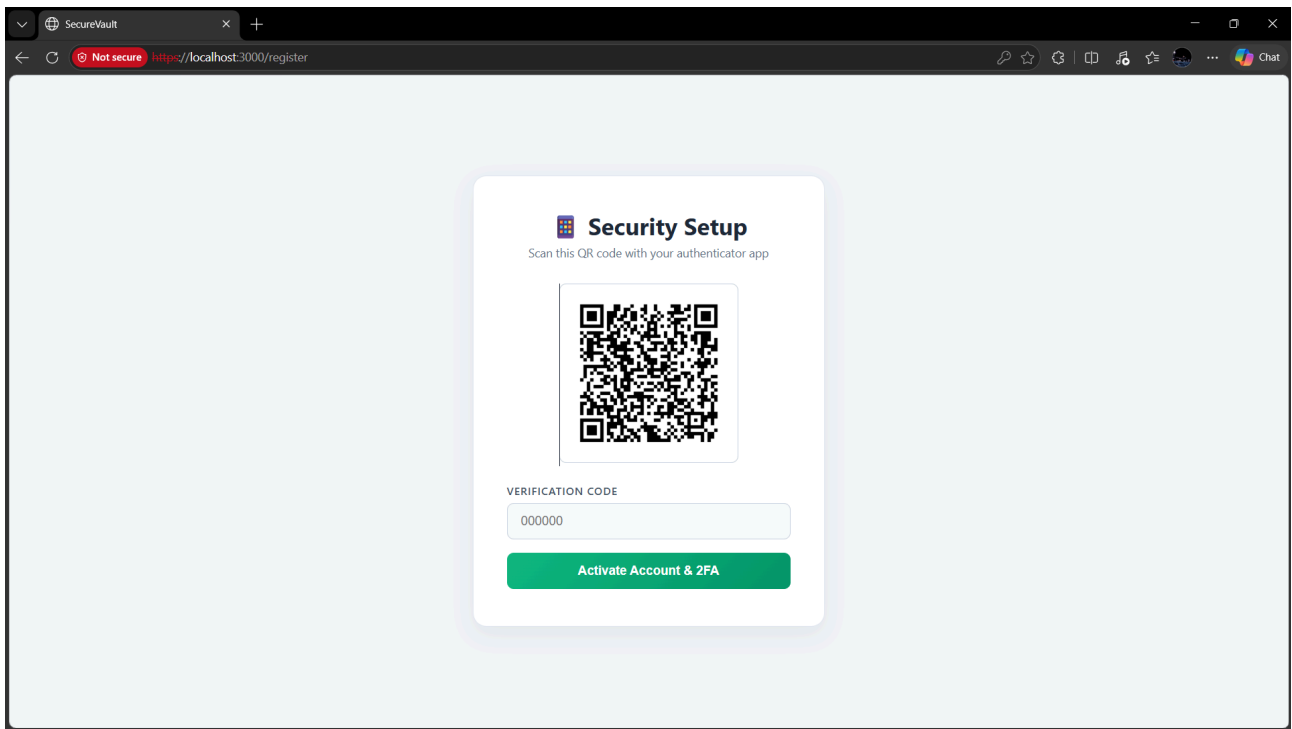


Figure 5: 2FA setup with QR code generation.

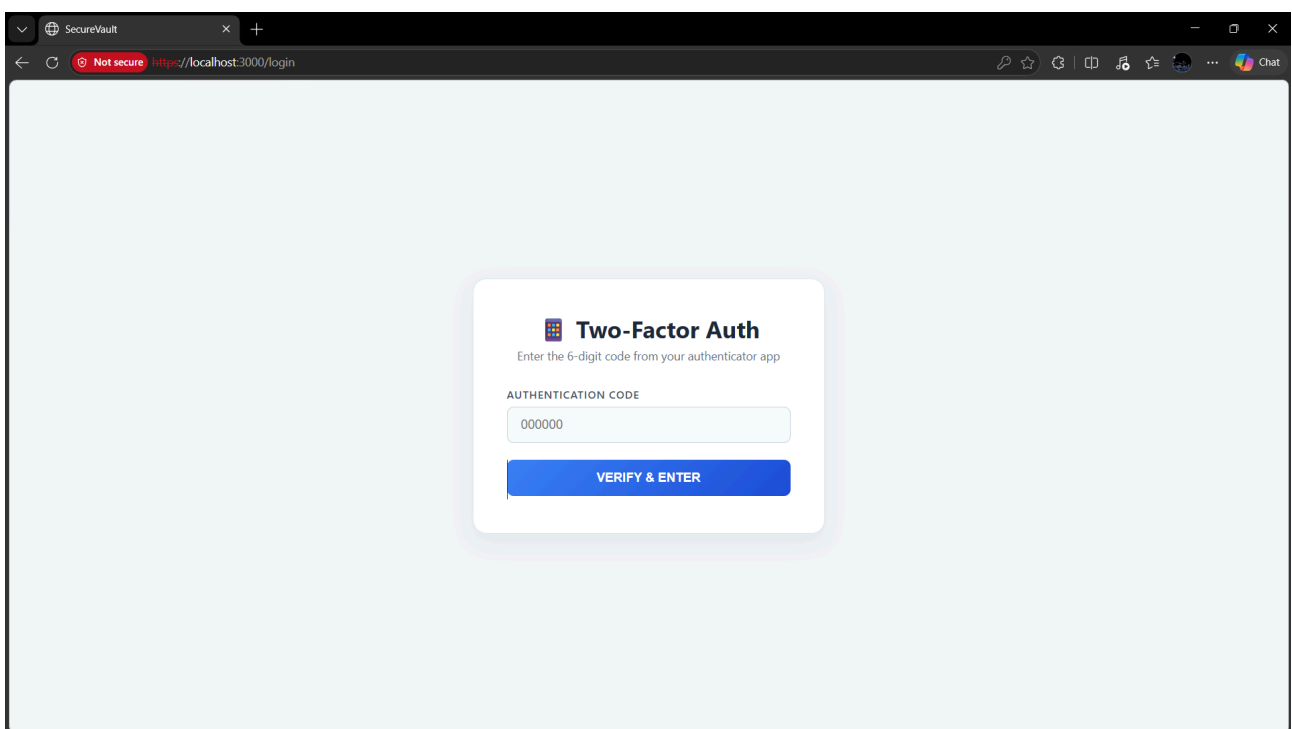


Figure 6: 2FA verification screen during login.

6.5 Admin Panel

Administrators can manage users, roles, and monitor system activity.

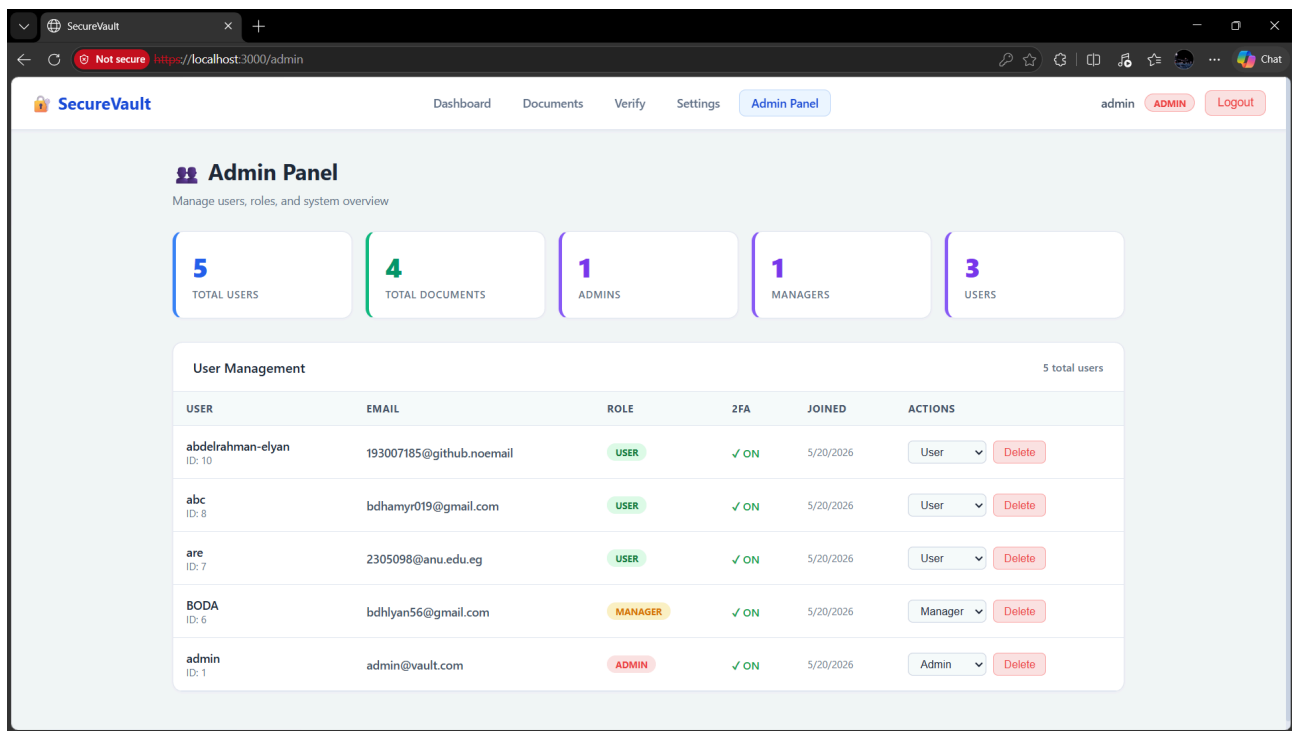


Figure 7: Administrative interface for user and role management.

7. Network Security Analysis: HTTPS vs HTTP

7.1 HTTP - Vulnerability Demonstration

When using unencrypted HTTP, sensitive data like credentials are transmitted in plain text, making them vulnerable to interception.

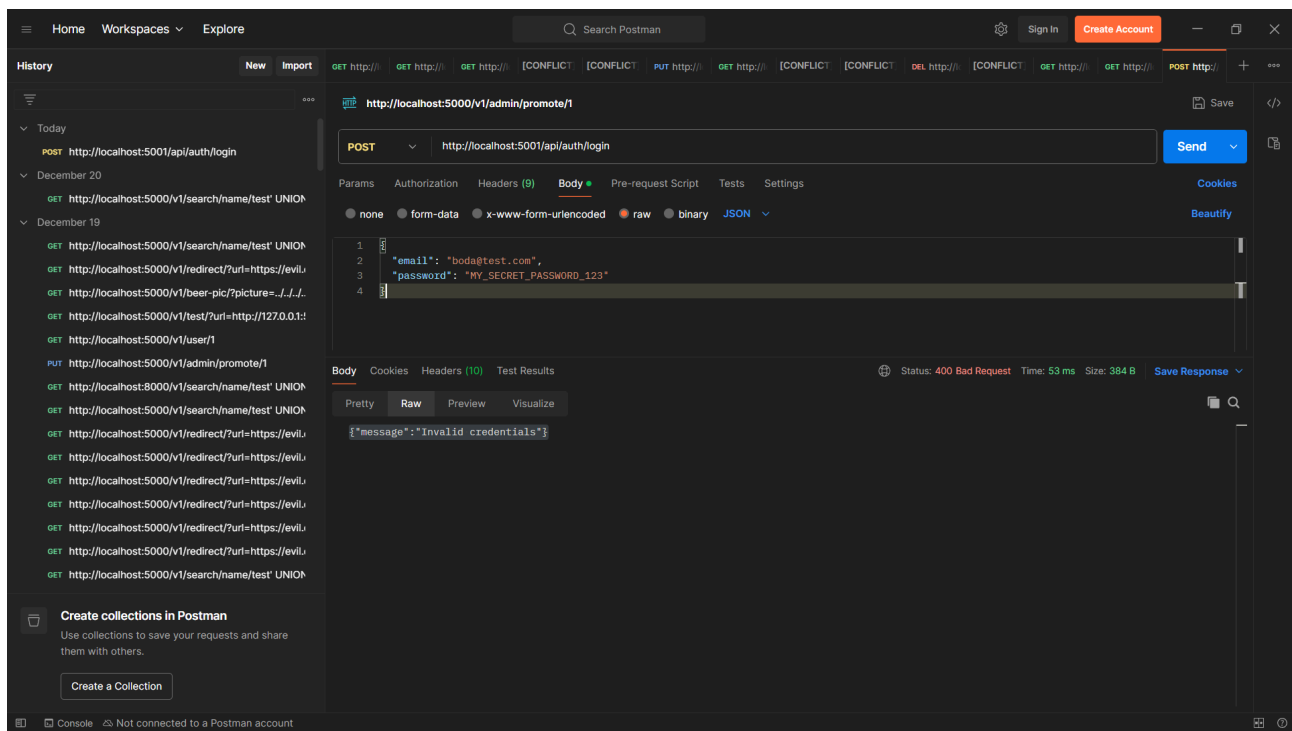


Figure 8: HTTP POST Request showing credentials in plain text in Postman.

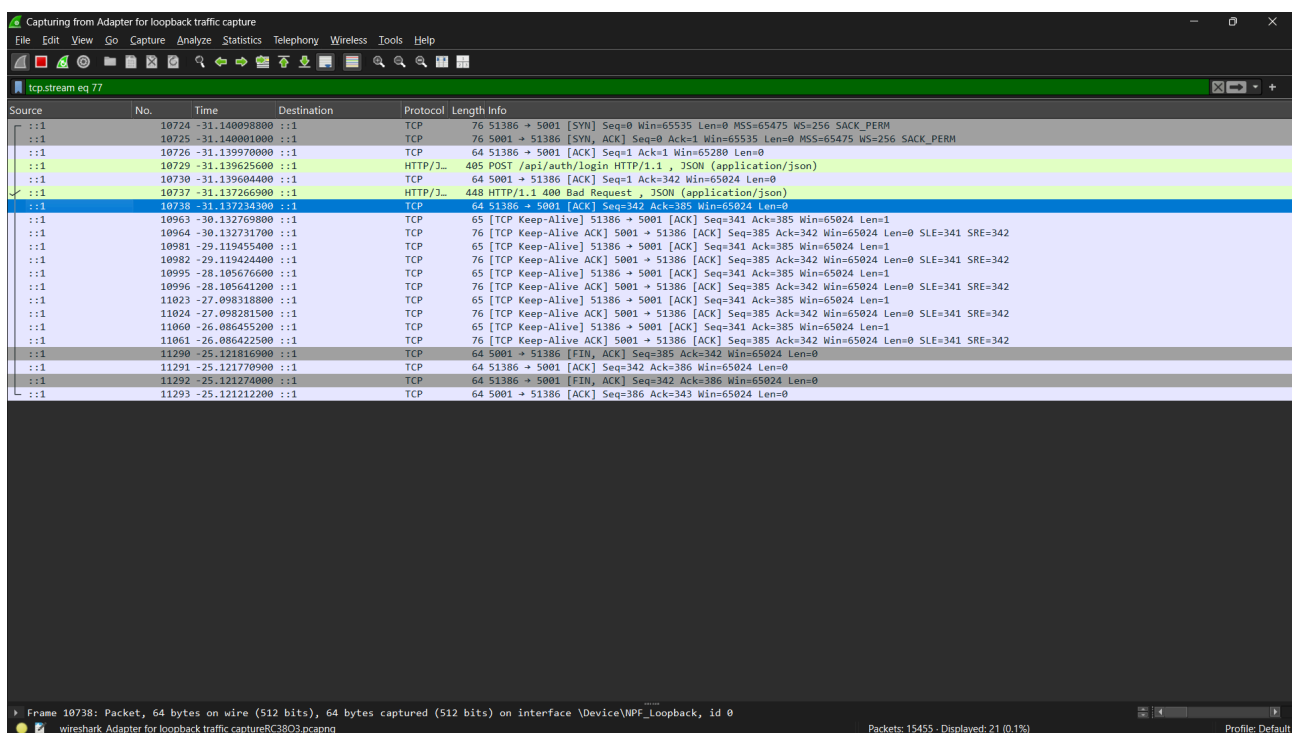
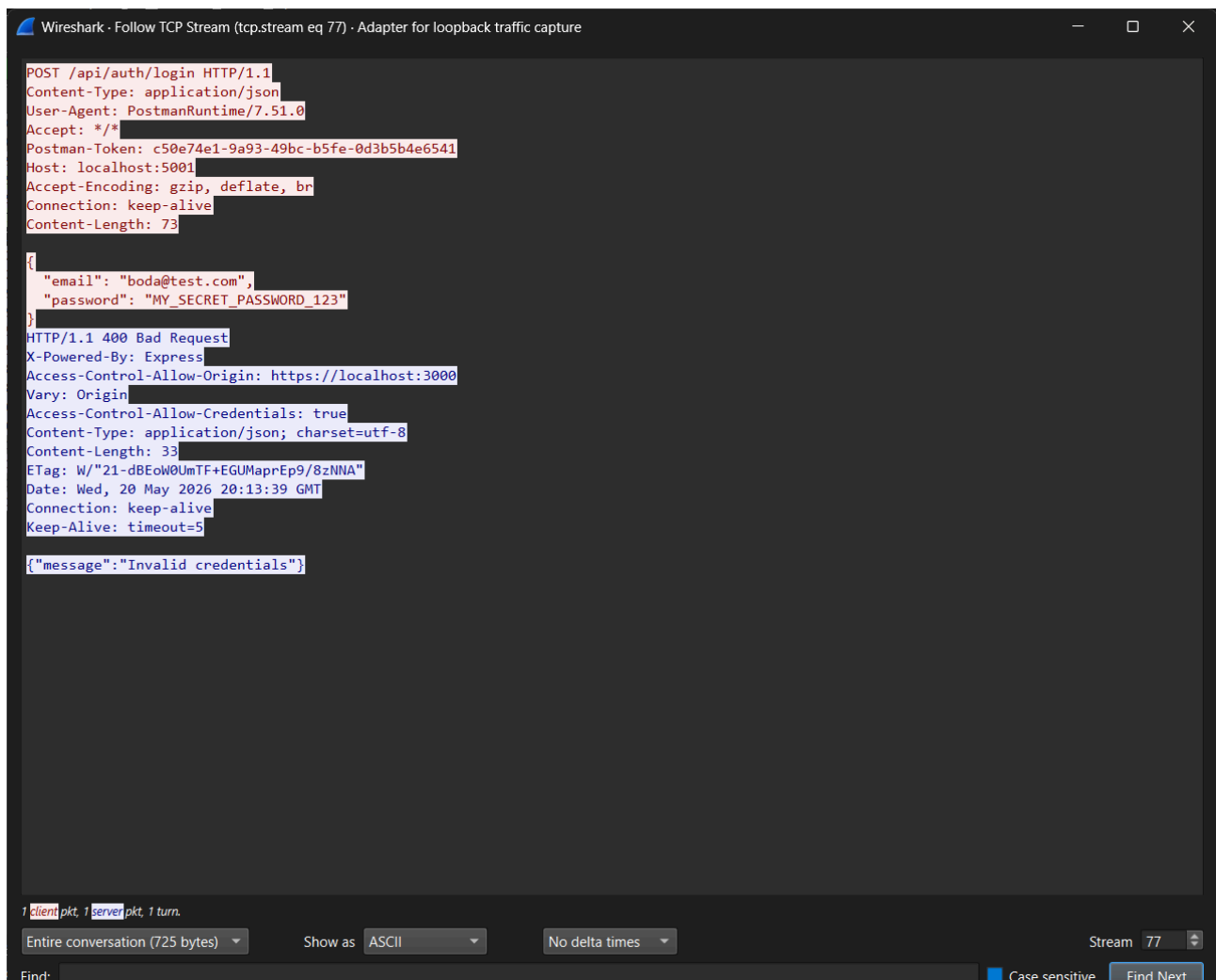


Figure 9: Wireshark capturing unencrypted HTTP traffic.



Wireshark · Follow TCP Stream (tcp.stream eq 77) · Adapter for loopback traffic capture

```
POST /api/auth/login HTTP/1.1
Content-Type: application/json
User-Agent: PostmanRuntime/7.51.0
Accept: */*
Postman-Token: c50e74e1-9a93-49bc-b5fe-0d3b5b4e6541
Host: localhost:5001
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Content-Length: 73

{
  "email": "boda@test.com",
  "password": "MY_SECRET_PASSWORD_123"
}

HTTP/1.1 400 Bad Request
X-Powered-By: Express
Access-Control-Allow-Origin: https://localhost:3000
Vary: Origin
Access-Control-Allow-Credentials: true
Content-Type: application/json; charset=utf-8
Content-Length: 33
ETag: W/"21-dBEoW0UmTF+EGUMaprEp9/8zNNA"
Date: Wed, 20 May 2026 20:13:39 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"message":"Invalid credentials"}
```

1 client pkt, 1 server pkt, 1 turn.

Entire conversation (725 bytes) Show as ASCII No delta times Stream 77

Find: Case sensitive Find Next

Figure 10: TCP Stream revealing plain text credentials (boda@test.com).

7.2 HTTPS - Secure Communication

HTTPS uses TLS to encrypt the communication channel, ensuring that even if traffic is intercepted, it remains unreadable.

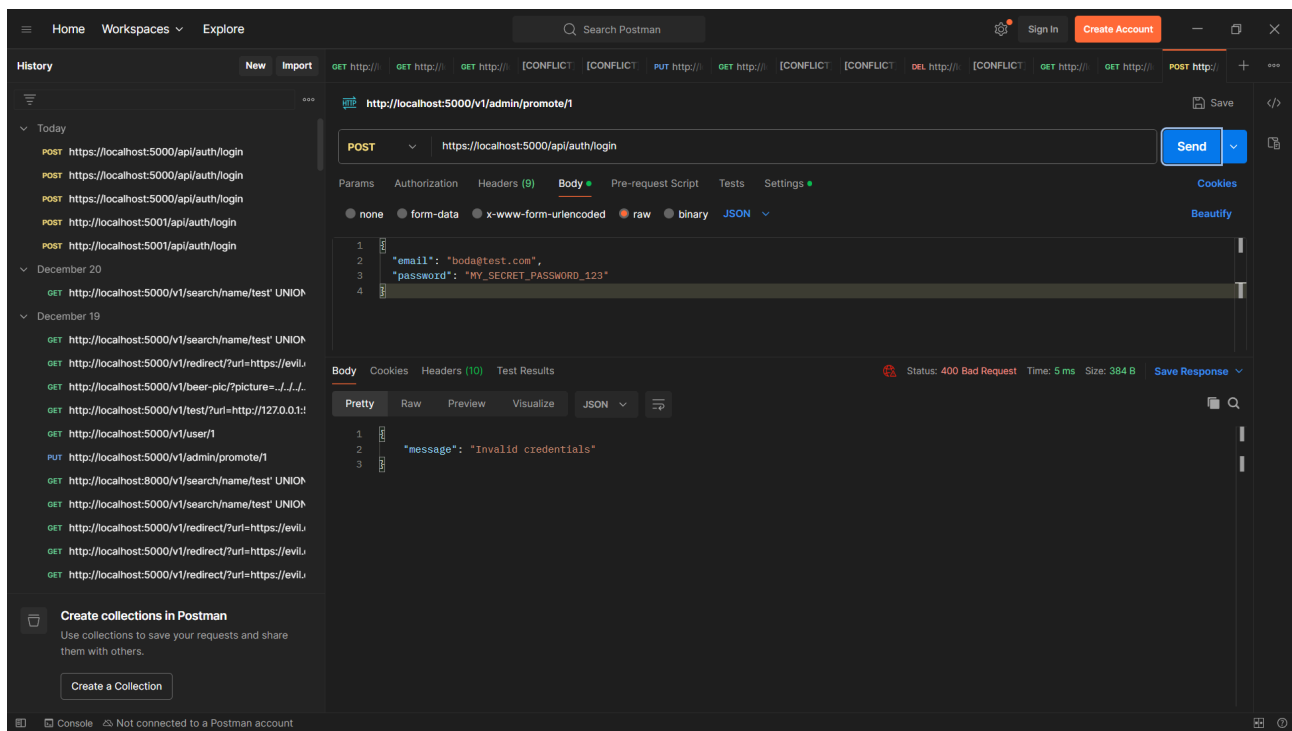


Figure 11: Secure HTTPS request in Postman.

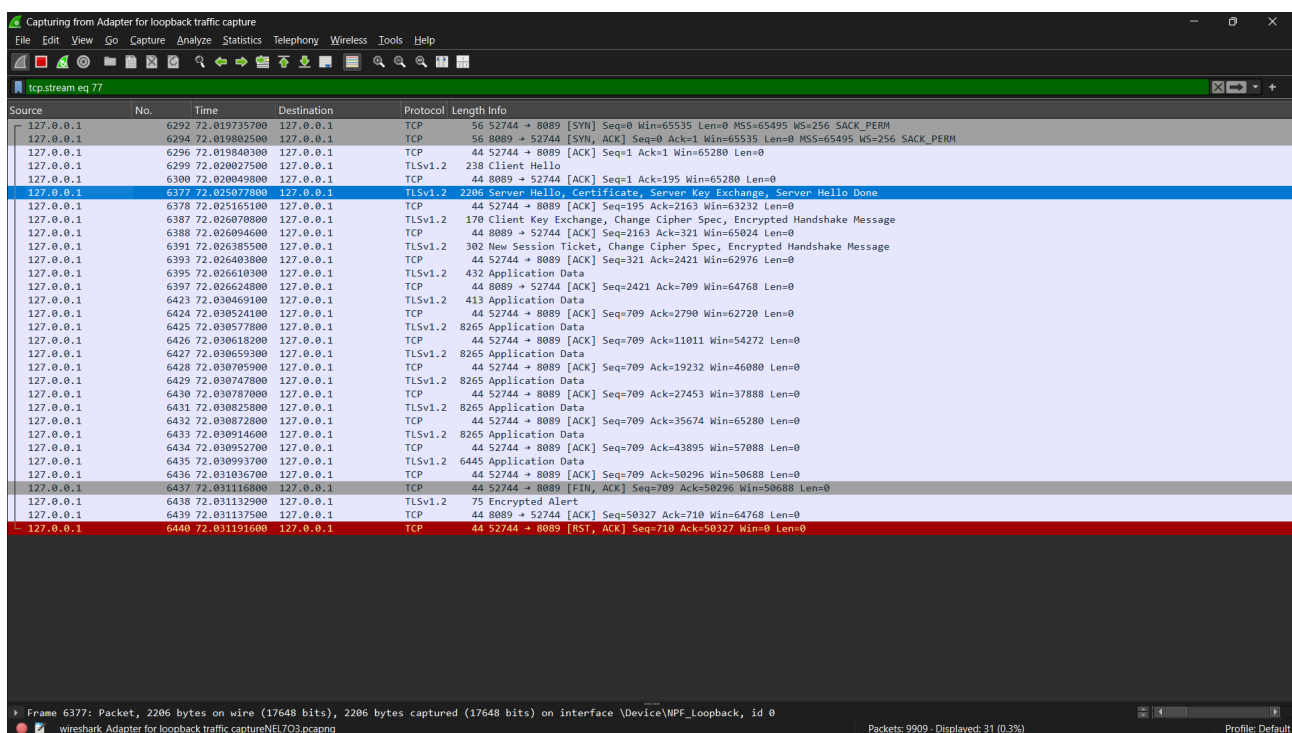


Figure 12: Wireshark showing TLS handshake and encrypted data transfer.

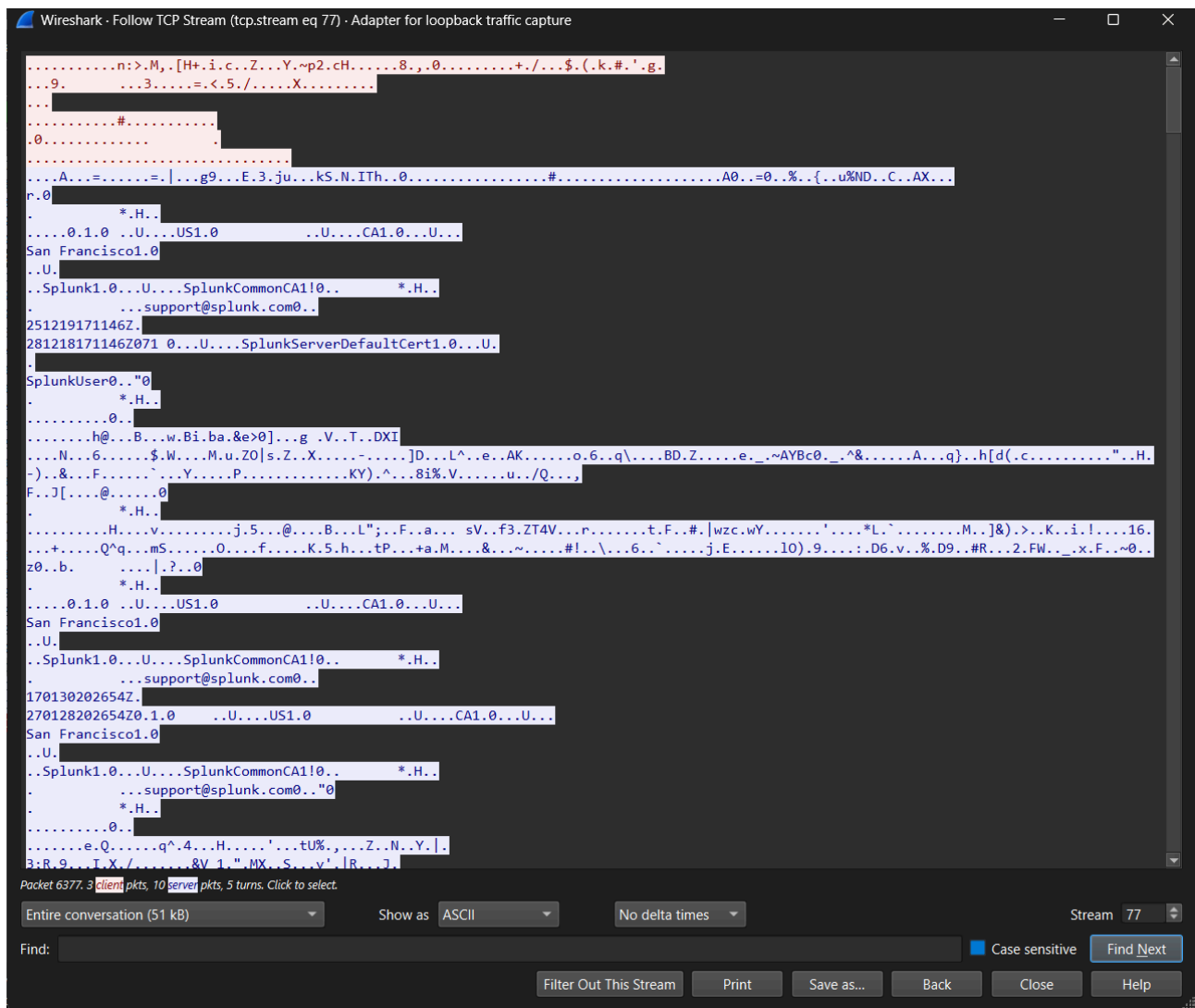


Figure 13: Encrypted binary data in an HTTPS stream, unreadable by attackers.

8. Authentication & Authorization Flow

8.1 Registration Process

1. User submits details (Username, Email, Password).
2. Backend validates input and password policy.
3. Password is hashed with bcrypt (salt rounds = 12).
4. 2FA is set up via QR code.
5. User record is created with default roles.

8.2 Login Process with 2FA

1. User enters credentials.
2. Server validates password hash and checks for account logout.
3. If 2FA is enabled, a temporary token is issued until the 6-digit code is verified.
4. Upon successful 2FA verification, a final JWT is issued for session management.

9. Conclusion

The Secure Document Vault project successfully demonstrates the integration of multiple security layers to protect sensitive data. By combining strong authentication (2FA, OAuth), robust encryption (AES-256, RSA), and secure communication (HTTPS), the system provides a high level of defense against common cyber threats. The network analysis clearly highlights the critical importance of HTTPS in protecting user credentials from interception.